

Dokumentáció: Bloch-gömb szimulátor

Kardos Máté (DP6MFQ), Kovács István (CQ7ZA1), Szakács Olivér (GAA9DD)

November 23, 2025

1 Áttekintés

A projektet egy webalkalmazásként valósítottuk meg JavaScript és WebGL alapokon (THREE.JS felhasználásával). A projekthez csatoltunk egy minimális webszerver alkalmazást, ami a localhost:8080 URL-en szolgálja ki az alkalmazást.

A projektet 3 fő részre osztottuk és ebben a felosztásban dokumentáltuk:

- Webes UI és kontroll logika
- Matematikai reprezentáció
- 3D-s megjelenítés

A kontroll logika felel azért, hogy összekösse a felhasználói inputokat (a kameramozgást kivéve) a háttérben lévő matematika reprezentációval és annak felhasználásával frissítse a 3D-s megjelenítést.

A matematika modell és a hozzá tartozó kód teljesen független a projekt többi részétől, csak a szükséges függvényeket és adatstruktúrákat szogáltatja.

A 3D-s megjelenítés szintén független a projekt maradékától, mivel csak egy API-t biztosít a szükséges elemek kirajzolásához. (Illetve, mivel a THREE.JS lehetővé teszi az egyszerű kameramozgatás implementálását, a kamera irányításának az inputjait is ez a komponens kezeli, megkerülve a kontrollert.)

2 Matematikai modell és implementáció

A kvantumállapotok reprezentációjára két módszert is használtunk. A megjelenítésnél előnyösebb a Descartes-féle derékszögű koordináta-rendszerben egy egységvektort használni, mivel azt egyszerű megjeleníteni. A kvantumkapuk esetében viszont az a könnyű, hogy ha csak egy mátrixot kell applikálni az állapotvektorra. Emiatt arra koncentráltunk, hogy a két állapot között tudjunk konvertálni és mindig a legegyszerűbb módszert használni.

2.1 Komplex számok

A kvantumállapotok és kapuk komplex amplitúdókkal dolgoznak. A megvalósításban minden komplex szám egy:

$$a = \{\text{re} : \alpha, \text{im} : \beta\}$$

objektum.

A fő műveletek:

- összeadás: $a + b$
- kivonás: $a - b$
- szorzás:

$$(a + ib)(c + id) = (ac - bd) + i(ad + bc)$$

- konjugálás: $a^* = \text{re} - i \text{im}$
- abszolút érték négyzete: $|a|^2 = \text{re}^2 + \text{im}^2$

Ezek építőkökként szolgálnak minden vektor- és mátrixművelethez.

2.2 Kvantumállapot reprezentáció

Egy egyqubites állapot a szokásos bázisban:

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

A programban ez a következő formában jelenik meg:

`[C(re_alpha,im_alpha), C(re_beta,im_beta)]`

2.3 Bloch-gömb konverzió

A Bloch-gömb egy valós 3D vektort rendel minden tiszta egyqubites állapothoz:

$$x = 2\Re(\alpha^*\beta), \quad y = 2\Im(\alpha^*\beta), \quad z = |\alpha|^2 - |\beta|^2.$$

A következőkben lépésről lépésre elmagyarázzuk, hogyan és miért működik ez a leképezés, valamint hogyan lehet stabilan implementálni a gyakorlatban.

Miért létezik ez a leképezés?

Egy egyqubites állapotot felírhatunk

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

A kvantummechanika szerint a globális fázis nem mérhető, azaz az állapotok közti különbség csak az amplitúdók relatív fázisának és arányának függvénye. Ezt a két szabadságfokot a Bloch-gömb pontja (x, y, z) kifejezi: a gömb két szöge (θ, ϕ) egyértelműen leírja az állapotot a globális fázis figyelmen kívül hagyásával.

Állapot $\rightarrow$ Bloch:

Számítsuk ki a Bloch-koordinátákat:

$$\alpha^*\beta = |\alpha||\beta|e^{i(\arg\beta - \arg\alpha)}.$$

Ennek a valós és képzetes részei adják $x/2$ és $y/2$. A z komponens a bázisállapotok foglaltságkülönbségét méri:

$$z = |\alpha|^2 - |\beta|^2.$$

Ezek az összefüggések közvetlenül vezetnek a `stateToBloch()` implementációhoz: először kiszámoljuk $\alpha^*\beta$ -t (komplex konjugál és szorzat), majd x, y, z -t a fenti formulák szerint, és visszaadjuk `THREE.Vector3(x, y, z)`-ként.

Bloch $\rightarrow$ állapot:

A Bloch-vektor pontjai a gömbön a szögekkel kapcsolatos egyszerű trigonometriai összefüggésekkel írhatók fel:

$$z = \cos \theta, \quad x = \sin \theta \cos \phi, \quad y = \sin \theta \sin \phi,$$

ahonnan

$$\theta = \arccos(z), \quad \phi = \text{atan2}(y, x).$$

Ezek után egy kényelmes és gyakran használt választás (a globális fázis rögzítése érdekében) a következő állapot:

$$\alpha = \cos \frac{\theta}{2}, \quad \beta = e^{i\phi} \sin \frac{\theta}{2}.$$

Ez kielégíti a normálási feltételt, és a relatív fázist ϕ -ként helyezi el a β komponensben.

Miért elég ez a formula?

A trigonometriai azonosságok miatt:

$$|\alpha|^2 = \cos^2 \frac{\theta}{2} = \frac{1 + \cos \theta}{2} = \frac{1 + z}{2},$$
$$|\beta|^2 = \sin^2 \frac{\theta}{2} = \frac{1 - \cos \theta}{2} = \frac{1 - z}{2},$$

és

$$2\Re(\alpha^* \beta) = 2 \cos \frac{\theta}{2} \sin \frac{\theta}{2} \cos \phi = \sin \theta \cos \phi = x,$$
$$2\Im(\alpha^* \beta) = \sin \theta \sin \phi = y.$$

Tehát a fenti α, β visszavezeti pontosan az eredeti x, y, z -et (globális fázis nélkül).

2.4 Mátrixszorzás és normalizálás

A kvantumkapuk 2×2 -es komplex mátrixok, amelyeket egy állapotra a szokásos mátrixszorzással alkalmazunk:

$$|\psi'\rangle = U|\psi\rangle.$$

A `matrixVectorMultiply()` függvény ezt általánosan kezeli. Minden eredmény végül normalizált lesz:

$$|\psi\rangle \leftarrow \frac{|\psi\rangle}{\sqrt{|\alpha|^2 + |\beta|^2}}.$$

2.5 Kvantumkapuk

Minden kapu alkalmazásának lépései:

1. Bloch-vektor visszaalakítása kvantumállapottá.
2. Mátrixszorzás a kapumátrixszal.
3. Normalizálás.
4. Visszakonvertálás Bloch-vektorra.

2.6 Kapuk alkalmazása Bloch-gömbre

A 3D vizualizáció csak Bloch-koordinátákkal dolgozik, ezért a kapuk **Bloch** $\rightarrow$ **állapot** $\rightarrow$ **kapu** $\rightarrow$ **állapot** $\rightarrow$ **Bloch** módon működnek.

Például az X-kapu:

$$(x, y, z) \xrightarrow{\text{Bloch} \rightarrow \text{state}} |\psi\rangle \xrightarrow{X} X|\psi\rangle \xrightarrow{\text{state} \rightarrow \text{Bloch}} (x', y', z').$$

Ez biztosítja, hogy a 3D vizualizáció és a matematikai modell mindig pontos összhangban legyen.

3 Webalkalmazás és 3Ds megjelenítés

A rövid fejlesztési idő érdekében a THREE.JS könyvtárt használtuk a 3Ds megjelenítéshez. A THREE.JS előnye, hogy sok grafikai primitívet tartalmaz, ami lehetővé teszi a projekt gyors felállítását. Ilyenek például a körívek (a gömb megjelenítéséhez), hengerek és kúpok (vektorok rajzolásához) illetve az orthografikus kamera és annak kontroller kódja.

3.1 Web

A THREE.JS miatt esett a választás a webes technológiára. Léteznek asztali alkalmazáshoz használható könyvtárak vagy játékmotorok is, de a könyvtárak általában bonyolultabbak mint a THREE, a játékmotorok esetében pedig úgy gonoltuk, hogy "ágyúval verébre" szituáció lenne, ha egy komplex motort használnánk egy ilyen egyszerű vizualizációhoz.

Bár egy egyszerű webes alkalmazást szerver nélkül is el tud indítani a bögésző, úgy gondoltuk, hogy robusztusabb megoldás lenne egy kisméretű szerverrel kiszolgálni az alkalmazást. Igyekeztünk minél kisebb és egyszerűbb megoldást találni, ami keresztplatformos is (bár a feladatkiírás csak a windows-t követelte meg, szükség volt linux támogatásra is, mert a csapat egy része azon fejlesztett).

A legjobb megoldásnak az bizonyult, hogy a go programozási nyelv standard könyvtárának HTTP szerver implementációját felhasználva építünk egy minimális szerveret. A szerver kódja megtalálható a server.go fájlban. A platformként lefordított futtathatókat beletettük a projektbe, így biztosítva, hogy a futtatásnak ne legyen függősége a go.

3.2 3D

A 3d megjelenítés fő részei a következők: Segédrács és Bloch-gömb illetve a vektorok. Ezen alakzatok kirajzolásán kívül még érdekes lehet a válaszottt koordináta rendszer illetve az árnyalás.

3.2.1 Koordináta rendszer

A THREE.JS egy jobbkezes koordinátarendszert használ, ahol az y tengely mutat felfele. Ez természetesen annyit jelent csak, hogy a koordinátarendszer jobbkezes és a kamera alapértelmezett UP vektora (0, 1, 0). A bloch gömb általában jobbkezes Z-up koordinátarendszerben van reprezentálva (legalábbis az előadásdiákban és más szimulátorokban így van). Nem lenne semmi akadálya, hogy egy Y-up rendszert használjunk, hiszen a

reprezentációra az is teljesen alkalmas, csak a konverziós matematika és az állapotvektor-Bloch-vektor megfeleltetések változnának meg (pl. $-0j -i$ (0,0,1) helyett (0,1,0) stb.).

Mindenesetre a konvenció megtartása mellett döntöttünk és inkább a kamera állását változtattuk meg (és a segédrácsot forgattuk el).

A koordinátarendszer és a nézet beállítását az orthografikus kamera paraméterezése határozza meg, ahol a **up** vektor Z-tengelyre állításával értük el a kívánt tájolást:

```
1 const camera = new THREE.OrthographicCamera(  
2   -orthoSize * aspect, orthoSize * aspect,  
3   orthoSize, -orthoSize,  
4   0.1, 1000  
5 );  
6 // A Z tengely mutat felfele a kamera számára  
7 camera.up.set(0,0,1);  
8 camera.position.set(-2.5, -2.5, 2.5);  
9 camera.lookAt(0,0,0);
```

3.2.2 Árnyalás

A felület nélküli alakzatok (szakaszok, ívek) egyszínű megjelenítést használtunk. A felületek esetében szintén egyszínű (árnyalás és árnyékolás nélküli) megjelenítést használtunk az egyszerűség kedvéért. (És egyébként is furcsa lett volna egy ilyen típusú megjelenítőben fényforrásokat definiálni.)

3.2.3 Segédrács és Bloch-gömb

A koordinátarendszer felfogását és a térérzet kialakulását jelentősen segíti a rács, amit az XY síkkal (vízszintes) tettünk párhuzamossá. A megjelenítéshez a THREE.JS előre megírt AxisHelper osztályát használtuk.

A Bloch-gömb megjelenítésére egy áttetsző szürke gömböt használtunk, szintén a THREE.JS leimplementált osztályát használva. A könnyebb interpretáció érdekében kirajzoltatunk 3 a tengelyekre merőleges (és azokkal megegyező színű) főkört. Ehhez saját függvényt írtunk, ami legenerálja a kör adott számú pontját, majd összeköti őket szakaszokkal. Elég magas felbontás esetén a hatás kör alakú lesz.

A főkörök kirajzolásához szükséges pontok generálását a **generateArcPoints** függvény végzi. A függvény a kezdőpontból kiindulva egy forgatási tengely (normálvektor) körül forgatja a vektort kvaterniók segítségével, így megkapva az ív pontjait:

```
1 function generateArcPoints(vStart, arcPlaneNormal, angle, numPoints) {  
2   if (numPoints <= 1) {  
3     return numPoints === 1 ? [vStart.clone()] : [];  
4   }  
5  
6   const axis = arcPlaneNormal.clone().normalize();  
7   const stepAngle = angle / (numPoints - 1);  
8  
9   // Forgatas kvaternioval  
10  const qStep = new THREE.Quaternion().setFromAxisAngle(axis, stepAngle  
11    );  
12  
13  const points = [];  
14  const currentPoint = vStart.clone();  
15  for (let i = 0; i < numPoints; i++) {
```

```

16     points.push(currentPoint.clone());
17     currentPoint.applyQuaternion(qStep);
18 }
19
20 return points;
21 }

```

Listing 1: A körív pontjainak generálása

3.2.4 Vektorok

A vektorok megjelenítéséhez saját függvényt írtunk, ami két meglévő alakzatot a THREE.js henger és kúpot használja fel a vektor szárának és csúcsának kirajzolására.

A vektorok megjelenítését a `drawVector` függvény végzi. A függvény dinamikusan hoz létre egy csoportot (`THREE.Group`), amely tartalmazza a vektor testét alkotó henger és a fejét alkotó kúpot. A csoportot végül a `setFromUnitVectors` segítségével forgatjuk a megfelelő irányba:

```

1 function drawVector(scene, vEnd, colorHex, options = {}) {
2     // ... opciók kibontása ...
3     const length = vEnd.length();
4     const arrowGroup = new THREE.Group();
5
6     // 1. A szár (henger) létrehozása
7     const shaftLength = length - headLength;
8     if (shaftLength > 0) {
9         const shaftGeometry = new THREE.CylinderGeometry(shaftRadius,
10 shaftRadius, shaftLength, 8);
11         const shaft = new THREE.Mesh(shaftGeometry, material);
12         shaft.position.y = shaftLength / 2;
13         arrowGroup.add(shaft);
14     }
15
16     // 2. A fej (kúp) létrehozása
17     const headGeometry = new THREE.ConeGeometry(headRadius, headLength,
18 12);
19     const head = new THREE.Mesh(headGeometry, material);
20     head.position.y = length - headLength / 2;
21     arrowGroup.add(head);
22
23     // 3. Forgatás a celirányba
24     const direction = vEnd.clone().normalize();
25     const upVector = new THREE.Vector3(0, 1, 0);
26     const quaternion = new THREE.Quaternion().setFromUnitVectors(
27 upVector, direction);
28
29     arrowGroup.setRotationFromQuaternion(quaternion);
30     scene.add(arrowGroup);
31     return arrowGroup;
32 }

```

Listing 2: Vektor kirajzolása és forgatása

4 UI és kontroll logika

4.1 Áttekintés

A rendszer két fő rétegből áll:

- **Adatréteg (Qbit modellek)**
- **UI réteg (DOM elemek + interakciók)**

A rétegeket folyamatos, kétirányú frissítés köti össze, amelyet főként a `refresh()` és különböző listenerek koordinálnak.

4.2 Adatréteg (Model) — Qbit

A **Qbit** a rendszer központi adatmodellje.

4.2.1 Főbb metódusok

- `refresh()` – gondoskodik a modell, a UI és a 3D scene szinkronizációjáról:
 1. Törli a régi 3D objektumot a scene-ből.
 2. Újrarajzolja a frissített vektort.
 3. Meghívja az UI frissítőt: `updateCoordinates(id, current)`.
- `reset()`, `remove()` és a kapu alkalmazása után mindig `refresh()` fut.

4.3 UI réteg — DOM elemek

A `createElement(...)` minden Qbithez létrehoz:

- Színdobozt
- Koordináta sort
- Reset és törlés gombot

Minden DOM elem az `id` alapján kapcsolódik a megfelelő Qbithoz a modellben. A DOM nem számol, csak kijelöl, indít, és tükrözi a modell változásait.

4.4 Listenerek

A listenerek a rendszer egyik legfontosabb részei, mivel összekapcsolják a felhasználói lépéseket a modell műveleteivel.

4.4.1 Regisztráció

A `RegisterListeners()` a betöltéskor hívódik meg, és az alábbi eseményeket köti:

- Kapu gombok (X, Y, Z, H, S, Phase)
- Add gomb
- Clear all gomb
- Téma váltó
- Koordináta inputok
- Reset/Törlés gombok és wrapper kattintások a DOM listaelemein

4.4.2 Model → UI visszacsatolás

Minden frissítés végén a listenerek **nem frissítik manuálisan a DOM-ot**. A modell maga kezeli a DOM frissítését, lentebb látható egy példa hogy hogyan:

- `refresh()` → `updateCoordinates(...)`

Így a UI soha nem írja felül hibásan a modell értékeit.

4.5 CSS rövid összefoglaló

A rendszer felhasználói felületét modern, két témás (light/dark) stílus jellemzi amiket futás közben tudunk változtatni:

- **Alap:** teljes képernyős layout, Arial betű, light/dark témák háttér- és színbeállításokkal.
- **UI konténer (#ui-container):** bal felső sarok, flex oszlop, lekerekített sarkok, árnyék, animált nyitás/bezárás.
- **Gombok:** .fancy-btn gradienttel, hover/active animáció, kapu gombok (X, Y, Z, H, S, PH input).
- **Listaelemek (#listContainer .element):** flex, lekerekített sarkok, témától függő színek, hover effekt, görgethető.
- **Műveletek:** .delete-btn és .reset-btn hover animáció, ikonok pointer-events: none.
- **Scene:** A Scene és a gömb a kiválasztott téma színeiben jelenik meg, és az egész képernyőt lefedi.